LOW LEVEL DESIGN DOCUMENT

Loan Management System

End-to-End Loan Web Application

TABLE OF CONTENTS

1. Introduction

2. Technology Stack Details

3. Class Design - Module Wise

4. Database Schema Details

5. API Specifications

6. Algorithm Implementations

7. Sequence Diagrams

8. Error Handling Strategy

9. Security Implementation

10. Performance Considerations

11. Gold Loan Implementation

1. INTRODUCTION

1.1 Purpose

This Low Level Design (LLD) document provides detailed technical specifications for implementing the Loan Management System. It defines classes, methods, data structures, algorithms, database schemas, API contracts, and implementation of details required for development.

1.2 Scope

This document covers:

• Detailed class designs with attributes, methods, and access modifiers
• Complete database schema with all tables, columns, constraints, and indexes
• API specifications with request/response formats and validation rules
• Core algorithm implementations (EMI calculation, FOIR, eligibility, etc.)
• Sequence diagrams for critical workflows
• Comprehensive error handling and exception hierarchy
• Security implementation (authentication, authorization, encryption)
• Performance optimization strategies
• Gold Loan specific implementation details

1.3 Key Design Principles

• SOLID Principles: Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion

• DRY (Don't Repeat Yourself): Code reusability through inheritance and composition

• Separation of Concerns: Clear boundaries between layers (Controller, Service, Repository)

• Fail-Fast: Early validation and exception handling

• Security by Design: Built-in security at every layer

2. TECHNOLOGY STACK DETAILS

2.1 Backend Technologies

Component	Technology	Version	Purpose
Framework	Spring Boot	3.2.x	Main application framework
Build Tool	Maven	3.9.x	Dependency management
Language	Java	17 LTS	Programming language
ORM	Hibernate (Spring Data JPA)	6.x	Object-relational mapping
Database	MySQL	8.0.x	Primary datastore
Security	Spring Security	6.x	Authentication & authorization
JWT	jjwt library	0.12.x	Token management
Validation	Hibernate Validator	8.x	Bean validation
Logging	SLF4J + Logback	Latest	Logging framework

2.2 Frontend Technologies

Component	Technology	Version	Purpose
Framework	React.js	18.x	UI framework
State Management	Redux Toolkit	2.x	Global state
Routing	React Router	6.x	Client-side routing
HTTP Client	Axios	1.6.x	API communication
CSS	Tailwind CSS	3.x	Utility-first styling
Forms	Formik + Yup	2.x / 1.x	Form handling & validation

3. CLASS DESIGN - MODULE WISE

3.1 Auth & User Management Module

3.1.1 User Entity

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long userId;

    @Column(unique = true, nullable = false, length = 255)
    private String email;
```

```
    @Column(nullable = false, length = 255)
    private String passwordHash;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private UserRole role;

    @Column(nullable = false)
    private Boolean isActive = true;

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;

    // Getters, Setters, Constructors
}
```

3.1.2 UserRole Enum

```
public enum UserRole {
    CUSTOMER("Customer"),
    LOAN_OFFICER("Loan Officer"),
    ADMIN("Administrator"),
    private final String displayName;

    UserRole(String displayName) {
        this.displayName = displayName;
    }

    public String getDisplayName() {
        return displayName;
    }
}
```

3.1.3 AuthService - Key Methods

Class: AuthService

Responsibility: Handle user authentication and token management

Methods:

Method	Parameters	Return Type	Description
register()	RegisterRequest	AuthResponse	Register new user, hash password, generate JWT
login()	LoginRequest	AuthResponse	Validate credentials, generate JWT

			token
refreshToken()	String token	AuthResponse	Validate and refresh JWT token
logout()	String token	void	Blacklist token to invalidate
requestPasswordReset()	String email	void	send via email

3.2 Customer Profile & KYC Module

3.2.1 Customer Entity

```
@Entity
@Table(name = "customers")
public class Customer {
    @Id
    private Long customerId; // Maps to userId

    @OneToOne
    @MapsId
    @JoinColumn(name = "customer_id")
    private User user;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    @Column(unique = true)
    private String phone;

    @Column(nullable = false)
    private LocalDate dateOfBirth;

    @Column(unique = true, length = 10)
    private String panNumber;

    @Column(unique = true, length = 12)
    private String aadhaarNumber; // Encrypted

    @Column(precision = 15, scale = 2)
    private BigDecimal annualIncome;

    @Enumerated(EnumType.STRING)
    private EmploymentType employmentType;

    private String addressStreet;
    private String addressCity;
    private String addressState;
    private String addressPincode;
```

```
    private String addressCountry = "India";

    @Enumerated(EnumType.STRING)
    private KycStatus kycStatus = KycStatus.PENDING;

    // Helper methods
    public int getAge() {
        return Period.between(dateOfBirth, LocalDate.now()).getYears();
    }

    public String getFullName() {
        return firstName + " " + lastName;
    }
}
```

3.2.2 KYC Entity

```
@Entity
@Table(name = "kyc")
public class Kyc {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long kycId;

    @Column(nullable = false, unique = true)
    private String applicationId;

    @Column(nullable = false)
    private Long customerId;

    @Enumerated(EnumType.STRING)
    private KycStatus status = KycStatus.PENDING;

    private String providerReference;

    @Column(columnDefinition = "TEXT")
    private String verificationDetails; // JSON

    private LocalDateTime initiatedAt;
    private LocalDateTime verifiedAt;
    private String failureReason;
    private Integer attemptCount = 0;
}
```

3.3 Loan Product Module

3.3.1 LoanProduct Entity

```
@Entity
@Table(name = "loan_products")
public class LoanProduct {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long productId;

    @Column(unique = true, nullable = false)
    private String productCode; // PL, BL, EL, VL, GL

    @Column(nullable = false)
    private String productName;

    @Column(precision = 15, scale = 2)
    private BigDecimal minLoanAmount;

    @Column(precision = 15, scale = 2)
    private BigDecimal maxLoanAmount;

    private Integer minTenureMonths;
    private Integer maxTenureMonths;

    @Column(precision = 5, scale = 2)
    private BigDecimal baseInterestRate; // Annual %

    @Column(precision = 5, scale = 2)
    private BigDecimal processingFeePercentage;

    private Boolean isSecured = false;

    @Column(precision = 5, scale = 2)
    private BigDecimal ltvRatio; // Loan-to-Value

    @Column(precision = 5, scale = 2)
    private BigDecimal foirLimit; // FOIR limit

    private Integer minCreditScore = 650;
    private Integer minAge = 21;
    private Integer maxAge = 65;

    @Column(precision = 15, scale = 2)
    private BigDecimal minMonthlyIncome;

    @Column(columnDefinition = "JSON")
    private String requiredDocuments;

    @Column(columnDefinition = "JSON")
    private String eligibilityRules;

    private Boolean isActive = true;
}
```

3.3.2 Product Configurations

Product	Code	Secured	LTV	FOIR	Special Rules
Personal Loan	PL	No	N/A	50%	Income-based
Business Loan	BL	No	N/A	60%	GST, vintage
Education Loan	EL	No	N/A	N/A	Co-applicant, moratorium
Vehicle Loan	VL	Yes	80%	50%	Down payment, valuation
Gold Loan	GL	Yes	75%	N/A	Gold valuation, no income

3.5 Loan Application Module

3.5.1 LoanApplication Entity

```
@Entity
@Table(name = "loan_applications")
public class LoanApplication {
    @Id
    @Column(unique = true, nullable = false, length = 20)
    private String applicationId; // APP-2026-00001

    @Column(nullable = false)
    private Long customerId;

    @Column(nullable = false)
    private Long productId;

    @Enumerated(EnumType.STRING)
    private ApplicationStatus status = ApplicationStatus.APPLIED;

    @Column(precision = 15, scale = 2, nullable = false)
    private BigDecimal requestedAmount;

    @Column(nullable = false)
    private Integer requestedTenureMonths;

    @Column(columnDefinition = "TEXT")
    private String purpose;

    @Column(precision = 15, scale = 2)
    private BigDecimal approvedAmount;

    private Integer approvedTenureMonths;
```

```
    @Column(precision = 5, scale = 2)
    private BigDecimal approvedInterestRate;

    @Column(precision = 15, scale = 2)
    private BigDecimal emiAmount;

    private String reviewedBy;
    private LocalDateTime reviewedAt;
    private String rejectionReason;

    @Column(columnDefinition = "JSON")
    private String additionalDetails;

    @PrePersist
    public void generateApplicationId() {
        if (this.applicationId == null) {
            int year = LocalDateTime.now().getYear();
            long seq = getNextSequenceValue();
            this.applicationId = String.format("APP-%d-%05d", year, seq);
        }
    }
}
```

3.5.2 ApplicationStatus Enum with State Transitions

```
public enum ApplicationStatus {
    APPLIED, KYC_PENDING, UNDER_REVIEW, APPROVED,
    REJECTED, DISBURSED, ACTIVE, OVERDUE, CLOSED;

    // State transition validation
    private static final Map<ApplicationStatus, Set<ApplicationStatus>>
        ALLOWED_TRANSITIONS = Map.of(
            APPLIED, Set.of(KYC_PENDING),
            KYC_PENDING, Set.of(UNDER_REVIEW, APPLIED),
            UNDER_REVIEW, Set.of(APPROVED, REJECTED),
            APPROVED, Set.of(DISBURSED),
            DISBURSED, Set.of(ACTIVE),
            ACTIVE, Set.of(OVERDUE, CLOSED),
            OVERDUE, Set.of(ACTIVE, CLOSED)
        );

    public boolean canTransitionTo(ApplicationStatus newStatus) {
        return ALLOWED_TRANSITIONS
            .getOrDefault(this, Set.of())
            .contains(newStatus);
    }
}
```

3.6 Credit Decisioning Module

3.6.1 EMI Calculation Algorithm

Standard EMI Formula Implementation:

```
/**
 * Calculate EMI using standard formula
 * EMI = P × r × (1+r)^n / ((1+r)^n - 1)
 *
 * Where:
 * P = Principal loan amount
 * r = Monthly interest rate (annual rate / 12 / 100)
 * n = Number of monthly installments
 *
 * Time Complexity: O(1) with power calculation
 * Space Complexity: O(1)
 */
public BigDecimal calculateEMI(
        BigDecimal principal,
        BigDecimal annualRate,
        int tenureMonths) {

    // Convert annual rate to monthly rate
    // r = annual_rate / 1200
    BigDecimal monthlyRate = annualRate
        .divide(new BigDecimal("1200"), 10, RoundingMode.HALF_UP);

    // Calculate (1 + r)
    BigDecimal onePlusR = BigDecimal.ONE.add(monthlyRate);

    // Calculate (1 + r)^n
    BigDecimal onePlusRPowerN = onePlusR.pow(tenureMonths);

    // Calculate numerator: P × r × (1+r)^n
    BigDecimal numerator = principal
        .multiply(monthlyRate)
        .multiply(onePlusRPowerN);

    // Calculate denominator: (1+r)^n - 1
    BigDecimal denominator = onePlusRPowerN.subtract(BigDecimal.ONE);

    // Calculate EMI
    BigDecimal emi = numerator.divide(
        denominator, 2, RoundingMode.HALF_UP
    );

    return emi;
}
```

```
// Example:
// Principal: ₹5,00,000
// Annual Rate: 12%
// Tenure: 36 months
// Monthly Rate: 12/1200 = 0.01
// (1+r)^n = 1.01^36 = 1.4307688
// EMI = 500000 × 0.01 × 1.4307688 / (1.4307688 - 1)
//     = 500000 × 0.01 × 1.4307688 / 0.4307688
//     = ₹16,607.54
```

3.6.2 FOIR Calculation Algorithm

Fixed Obligations to Income Ratio (FOIR) Calculation:

```
/**
 * Calculate FOIR (Fixed Obligations to Income Ratio)
 * FOIR = (Total Fixed Obligations / Monthly Income) × 100
 *
 * Fixed Obligations include:
 * - Existing loan EMIs
 * - Credit card minimum payments
 * - Other recurring obligations
 *
 * Returns: FOIR percentage
 */
private BigDecimal calculateFOIR(
        Customer customer,
        LoanApplication application) {

    // Get monthly income
    BigDecimal monthlyIncome = customer.getAnnualIncome()
        .divide(new BigDecimal("12"), 2, RoundingMode.HALF_UP);

    // Calculate proposed EMI
    LoanProduct product = productRepository
        .findById(application.getProductId())
        .orElseThrow();

    BigDecimal proposedEmi = calculateEMI(
        application.getRequestedAmount(),
        product.getBaseInterestRate(),
        application.getRequestedTenureMonths()
    );

    // Get existing obligations from credit report
    BigDecimal existingObligations =
        getExistingObligationsFromCreditReport(application);
```

```
    // Total obligations
    BigDecimal totalObligations =
        existingObligations.add(proposedEmi);

    // FOIR percentage = (Total Obligations / Income) × 100
    BigDecimal foir = totalObligations
        .divide(monthlyIncome, 4, RoundingMode.HALF_UP)
        .multiply(new BigDecimal("100"));

    return foir;
}

// Example:
// Monthly Income: ₹80,000
// Existing Obligations: ₹15,000
// Proposed EMI: ₹20,000
// Total Obligations: ₹35,000
// FOIR = (35000 / 80000) × 100 = 43.75%
```

4. DATABASE SCHEMA DETAILS

4.1 Complete Table Definitions

4.1.1 users Table

```
CREATE TABLE users (
    user_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('CUSTOMER', 'LOAN_OFFICER', 'ADMIN', 'COLLECTOR')
        NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,

    INDEX idx_email (email),
    INDEX idx_role (role),
    INDEX idx_active (is_active)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.2 customers Table

```
CREATE TABLE customers (
    customer_id BIGINT PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone VARCHAR(15) UNIQUE,
    date_of_birth DATE NOT NULL,
    pan_number VARCHAR(10) UNIQUE,
```

```
    aadhaar_number VARCHAR(12) UNIQUE COMMENT 'AES-256 encrypted',
    annual_income DECIMAL(15,2),
    employment_type ENUM('SALARIED', 'SELF_EMPLOYED', 'BUSINESS',
        'PROFESSIONAL', 'RETIRED', 'STUDENT'),
    address_street VARCHAR(200),
    address_city VARCHAR(100),
    address_state VARCHAR(100),
    address_pincode VARCHAR(10),
    address_country VARCHAR(50) DEFAULT 'India',
    kyc_status ENUM('PENDING', 'IN_PROGRESS', 'VERIFIED',
        'FAILED', 'EXPIRED') NOT NULL DEFAULT 'PENDING',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (customer_id) REFERENCES users(user_id)
        ON DELETE CASCADE,
    INDEX idx_phone (phone),
    INDEX idx_pan (pan_number),
    INDEX idx_kyc_status (kyc_status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.3 loan_applications Table

```
CREATE TABLE loan_applications (
    application_id VARCHAR(20) PRIMARY KEY,
    customer_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,
    status ENUM('APPLIED', 'KYC_PENDING', 'UNDER_REVIEW',
        'APPROVED', 'REJECTED', 'DISBURSED',
        'ACTIVE', 'OVERDUE', 'CLOSED') NOT NULL DEFAULT 'APPLIED',
    requested_amount DECIMAL(15,2) NOT NULL,
    requested_tenure_months INT NOT NULL,
    purpose TEXT,
    approved_amount DECIMAL(15,2),
    approved_tenure_months INT,
    approved_interest_rate DECIMAL(5,2),
    emi_amount DECIMAL(15,2),
    reviewed_by VARCHAR(100),
    reviewed_at TIMESTAMP,
    rejection_reason VARCHAR(500),
    additional_details JSON
        COMMENT 'Product-specific additional data',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES loan_products(product_id),
    INDEX idx_customer_status (customer_id, status),
```

```
    INDEX idx_product_status (product_id, status),
    INDEX idx_status (status),
    INDEX idx_created_at (created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.5 loans Table

```
CREATE TABLE loans (
    loan_account_number VARCHAR(20) PRIMARY KEY,
    application_id VARCHAR(20) NOT NULL UNIQUE,
    customer_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,
    sanctioned_amount DECIMAL(15,2) NOT NULL,
    interest_rate DECIMAL(5,2) NOT NULL,
    tenure_months INT NOT NULL,
    emi_amount DECIMAL(15,2) NOT NULL,
    outstanding_principal DECIMAL(15,2) NOT NULL,
    status ENUM('ACTIVE', 'OVERDUE', 'CLOSED')
        NOT NULL DEFAULT 'ACTIVE',
    start_date DATE NOT NULL,
    maturity_date DATE NOT NULL,
    next_emi_date DATE,
    total_paid DECIMAL(15,2) DEFAULT 0.00,
    total_pending DECIMAL(15,2),
    disbursement_date TIMESTAMP,
    closure_date TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (application_id)
        REFERENCES loan_applications(application_id),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES loan_products(product_id),
    INDEX idx_customer_status (customer_id, status),
    INDEX idx_status (status),
    INDEX idx_next_emi_date (next_emi_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.6 repayment_schedule Table

```
CREATE TABLE repayment_schedule (
    schedule_id BIGINT PRIMARY KEY AUTO_INCREMENT,
    loan_account_number VARCHAR(20) NOT NULL,
    installment_number INT NOT NULL,
    due_date DATE NOT NULL,
    emi_amount DECIMAL(15,2) NOT NULL,
    principal_component DECIMAL(15,2) NOT NULL,
    interest_component DECIMAL(15,2) NOT NULL,
    opening_balance DECIMAL(15,2) NOT NULL,
    closing_balance DECIMAL(15,2) NOT NULL,
```

```
    amount_paid DECIMAL(15,2) DEFAULT 0.00,
    status ENUM('PENDING', 'PAID', 'OVERDUE', 'WAIVED')
        NOT NULL DEFAULT 'PENDING',
    paid_date TIMESTAMP,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (loan_account_number)
        REFERENCES loans(loan_account_number),
    UNIQUE KEY unique_installment
        (loan_account_number, installment_number),
    INDEX idx_loan_due_date (loan_account_number, due_date),
    INDEX idx_status (status),
    INDEX idx_due_date (due_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.2 Indexing Strategy

Purpose: Optimize query performance for common access patterns

Table	Index Name	Columns	Purpose
users	idx_email	email	Fast login lookup
customers	idx_pan	pan_number	Unique identification
customers	idx_phone	phone	Contact lookup
loan_applications	idx_customer_status	customer_id, status	Customer loan queries
loan_applications	idx_created_at	created_at	Time-based reports
loans	idx_next_emi_date	next_emi_date	Due payment reminders
repayment_schedule	idx_loan_due_date	loan_account_number, due_date	Payment schedules

5. API SPECIFICATIONS

5.1 Standard Response Format

```
// All APIs return responses in this format
public class ApiResponse<T> {
    private String status;      // "SUCCESS" or "ERROR"
    private String message;     // Human-readable message
    private T data;             // Response payload
    private LocalDateTime timestamp;
    private List<String> errors; // Validation errors (if any)

    public static <T> ApiResponse<T> success(String msg, T data) {
        ApiResponse<T> response = new ApiResponse<>();
        response.setStatus("SUCCESS");
        response.setMessage(msg);
```

```
        response.setData(data);
        response.setTimestamp(LocalDateTime.now());
        return response;
    }

    public static <T> ApiResponse<T> error(String msg, List<String> errors) {
        ApiResponse<T> response = new ApiResponse<>();
        response.setStatus("ERROR");
        response.setMessage(msg);
        response.setErrors(errors);
        response.setTimestamp(LocalDateTime.now());
        return response;
    }
}
```

6. ALGORITHM IMPLEMENTATIONS

6.1 Amortization Schedule Generation

Algorithm to generate complete EMI repayment schedule:

```
/**
 * Generate amortization schedule for a loan
 *
 * Algorithm:
 * For each installment from 1 to n:
 *   1. Calculate interest for current month =
 *      opening_balance × monthly_rate
 *   2. Calculate principal component = EMI - interest
 *   3. Calculate closing balance =
 *      opening_balance - principal
 *   4. Create schedule entry
 *   5. Next opening balance = current closing balance
 *
 * Time Complexity: O(n) where n = tenure months
 * Space Complexity: O(n) for storing schedule
 */
public List<RepaymentSchedule> generateSchedule(
        String loanAccountNumber,
        BigDecimal principal,
        BigDecimal annualRate,
        int tenureMonths,
        LocalDate startDate) {

    List<RepaymentSchedule> schedule = new ArrayList<>();

    // Calculate monthly rate
```

```
    BigDecimal monthlyRate = annualRate
        .divide(new BigDecimal("1200"), 10, RoundingMode.HALF_UP);

    // Calculate fixed EMI
    BigDecimal emi = calculateEMI(principal, annualRate, tenureMonths);

    BigDecimal openingBalance = principal;
    LocalDate dueDate = startDate.plusMonths(1);

    for (int installmentNumber = 1;
         installmentNumber <= tenureMonths;
         installmentNumber++) {

        // Calculate interest component
        BigDecimal interestComponent = openingBalance
            .multiply(monthlyRate)
            .setScale(2, RoundingMode.HALF_UP);

        // Calculate principal component
        BigDecimal principalComponent = emi
            .subtract(interestComponent)
            .setScale(2, RoundingMode.HALF_UP);

        // For last installment, adjust principal to clear balance
        if (installmentNumber == tenureMonths) {
            principalComponent = openingBalance;
            emi = principalComponent.add(interestComponent);
        }

        // Calculate closing balance
        BigDecimal closingBalance = openingBalance
            .subtract(principalComponent)
            .setScale(2, RoundingMode.HALF_UP);

        // Create schedule entry
        RepaymentSchedule entry = new RepaymentSchedule();
        entry.setLoanAccountNumber(loanAccountNumber);
        entry.setInstallmentNumber(installmentNumber);
        entry.setDueDate(dueDate);
        entry.setEmiAmount(emi);
        entry.setPrincipalComponent(principalComponent);
        entry.setInterestComponent(interestComponent);
        entry.setOpeningBalance(openingBalance);
        entry.setClosingBalance(closingBalance);
        entry.setStatus(ScheduleStatus.PENDING);

        schedule.add(entry);

        // Update for next iteration
```

```
        openingBalance = closingBalance;
        dueDate = dueDate.plusMonths(1);
    }

    return schedule;
}
```

Example Schedule (First 3 Months):

Month	Opening	EMI	Principal	Interest	Closing
1	5,00,000	16,608	11,608	5,000	4,88,392
2	4,88,392	16,608	11,724	4,884	4,76,668
3	4,76,668	16,608	11,841	4,767	4,64,827

7. SEQUENCE DIAGRAMS

7.1 Gold Loan Application Workflow

Complete flow from application to disbursement:

Step 1: Customer Submission

→ Customer fills gold loan application form

→ Enters gold details (weight, purity, items)

→ System calculates eligible amount

→ Application created with status: APPLIED

Step 2: KYC Initiation

→ System checks if customer KYC verified

→ If not verified, status: KYC_PENDING

→ Customer uploads identity documents

→ External KYC provider validates

Step 3: Gold Verification

→ Application moves to UNDER_REVIEW

→ Customer brings gold items to branch

→ Valuation officer physically verifies:

- Weighs gold items
- Tests purity
- Confirms authenticity

→ Officer updates system with verified details

→ Gold marked as VERIFIED

Step 4: Eligibility Check

→ System runs eligibility engine

→ For gold loans: Only age check (no credit score)

→ Calculates final eligible amount

→ Generates loan offer

Step 5: Approval

→ Customer accepts offer

→ Loan officer reviews and approves

→ Status changes to APPROVED

→ Loan agreement generated

Step 6: Gold Pledging

→ Gold items physically pledged

→ Stored in designated locker/vault

→ Pledge receipt generated

→ Gold status: VERIFIED (pledged)

Step 7: Disbursement

→ Funds transferred to customer account

→ Loan activated

→ EMI schedule generated

→ Status: ACTIVE

Step 8: Repayment & Closure

→ Customer pays EMIs monthly

→ On full repayment or pre-closure:

- Loan marked CLOSED
- Gold items released
- Release receipt issued
- Gold status: RELEASED

8. ERROR HANDLING STRATEGY

8.1 Exception Hierarchy

```
// Base application exception
public class LmsException extends RuntimeException {
    private String errorCode;
    private HttpStatus httpStatus;

    public LmsException(String message, String errorCode,
                        HttpStatus httpStatus) {
        super(message);
        this.errorCode = errorCode;
        this.httpStatus = httpStatus;
    }
}

// Specific exceptions
public class ResourceNotFoundException extends LmsException {
    public ResourceNotFoundException(String resource) {
        super(resource + " not found",
             "RESOURCE_NOT_FOUND", HttpStatus.NOT_FOUND);
    }
}

public class ValidationException extends LmsException {
    public ValidationException(String message) {
        super(message, "VALIDATION_ERROR",
             HttpStatus.BAD_REQUEST);
    }
}

public class BusinessException extends LmsException {
    public BusinessException(String message) {
        super(message, "BUSINESS_RULE_VIOLATION",
             HttpStatus.UNPROCESSABLE_ENTITY);
    }
}
```

```
public class InsufficientFundsException extends LmsException {
    public InsufficientFundsException() {
        super("Insufficient funds for transaction",
             "INSUFFICIENT_FUNDS", HttpStatus.PAYMENT_REQUIRED);
    }
}
```

8.2 Global Exception Handler

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(LmsException.class)
    public ResponseEntity<ApiResponse<Void>> handleLmsException(
            LmsException ex) {
        return ResponseEntity
            .status(ex.getHttpStatus())
            .body(ApiResponse.error(ex.getMessage(),
                 List.of(ex.getErrorCode())));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiResponse<Void>> handleValidationErrors(
            MethodArgumentNotValidException ex) {
        List<String> errors = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(error -> error.getField() + ": " +
                         error.getDefaultMessage())
            .collect(Collectors.toList());

        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body(ApiResponse.error("Validation failed", errors));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse<Void>> handleGenericException(
            Exception ex) {
        // Log full stack trace
        logger.error("Unexpected error", ex);

        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(ApiResponse.error("Internal server error",
                 List.of("INTERNAL_ERROR")));
    }
}
```

9. SECURITY IMPLEMENTATION

9.1 JWT Token Implementation

```
@Component
public class JwtTokenProvider {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private long expirationTime; // 15 minutes in milliseconds

    public String generateToken(User user) {
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + expirationTime);

        return Jwts.builder()
            .setSubject(String.valueOf(user.getUserId()))
            .claim("email", user.getEmail())
            .claim("role", user.getRole().name())
            .setIssuedAt(now)
            .setExpiration(expiryDate)
            .signWith(SignatureAlgorithm.HS512, secret)
            .compact();
    }

    public Long getUserIdFromToken(String token) {
        Claims claims = Jwts.parser()
            .setSigningKey(secret)
            .parseClaimsJws(token)
            .getBody();

        return Long.parseLong(claims.getSubject());
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser().setSigningKey(secret).parseClaimsJws(token);
            return true;
        } catch (SignatureException | MalformedJwtException |
                 ExpiredJwtException | UnsupportedJwtException |
                 IllegalArgumentException ex) {
            return false;
        }
    }
}
```

9.2 Password Encryption

```
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        // BCrypt with strength 12
        return new BCryptPasswordEncoder(12);
    }
}

// Usage in service
public void createUser(RegisterRequest request) {
    User user = new User();
    user.setEmail(request.getEmail());

    // Hash password before storing
    String hashedPassword = passwordEncoder.encode(
        request.getPassword());
    user.setPasswordHash(hashedPassword);

    userRepository.save(user);
}

// Verification
public boolean verifyPassword(String rawPassword,
                              String hashedPassword) {
    return passwordEncoder.matches(rawPassword, hashedPassword);
}
```

9.3 Sensitive Data Encryption

Aadhaar and bank account numbers are encrypted at rest using AES-256:

```
@Component
public class EncryptionService {

    @Value("${encryption.key}")
    private String encryptionKey;

    private static final String ALGORITHM = "AES/GCM/NoPadding";

    public String encrypt(String plaintext) throws Exception {
        SecretKeySpec keySpec = new SecretKeySpec(
            encryptionKey.getBytes(), "AES");

        Cipher cipher = Cipher.getInstance(ALGORITHM);
        cipher.init(Cipher.ENCRYPT_MODE, keySpec);
```

```
        byte[] encrypted = cipher.doFinal(plaintext.getBytes());
        return Base64.getEncoder().encodeToString(encrypted);
    }

    public String decrypt(String ciphertext) throws Exception {
        SecretKeySpec keySpec = new SecretKeySpec(
            encryptionKey.getBytes(), "AES");

        Cipher cipher = Cipher.getInstance(ALGORITHM);
        cipher.init(Cipher.DECRYPT_MODE, keySpec);

        byte[] decoded = Base64.getDecoder().decode(ciphertext);
        byte[] decrypted = cipher.doFinal(decoded);
        return new String(decrypted);
    }
```

10. PERFORMANCE CONSIDERATIONS

10.1 Database Optimization

Caching Strategy:

• Loan product configurations cached with @Cacheable

• Cache eviction on product updates

• TTL: 1 hour for product data

```
@Service
public class LoanProductService {

    @Cacheable(value = "loanProducts", key = "#productCode")
    public LoanProduct getProductByCode(String productCode) {
        return productRepository.findByProductCode(productCode)
            .orElseThrow(() -> new ResourceNotFoundException(
                "Product not found"));
    }

    @CacheEvict(value = "loanProducts", allEntries = true)
    public LoanProduct updateProduct(Long productId,
                                     LoanProduct updates) {
        // Update logic
        return productRepository.save(updates);
    }
}
```

Query Optimization:

• Use @Query with JPQL for complex queries

• Fetch strategies: LAZY for collections

• Pagination for large result sets

```
@Repository
public interface LoanApplicationRepository
        extends JpaRepository<LoanApplication, String> {

    // Optimized query with pagination
    @Query("SELECT la FROM LoanApplication la " +
           "WHERE la.customerId = :customerId " +
           "AND la.status IN :statuses " +
           "ORDER BY la.createdAt DESC")
    Page<LoanApplication> findByCustomerAndStatuses(
        @Param("customerId") Long customerId,
        @Param("statuses") List<ApplicationStatus> statuses,
        Pageable pageable);

    // Count query for dashboard (faster than full fetch)
    @Query("SELECT COUNT(la) FROM LoanApplication la " +
           "WHERE la.status = :status")
    long countByStatus(@Param("status") ApplicationStatus status);
}
```

10.2 Async Processing

Long-running operations executed asynchronously:

```
@Service
public class CreditDecisionService {

    @Async
    public CompletableFuture<EligibilityResult>
            evaluateEligibilityAsync(String applicationId) {

        EligibilityResult result = evaluateEligibility(applicationId);
        return CompletableFuture.completedFuture(result);
    }
}

@Configuration
@EnableAsync
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(10);
```

```
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
        executor.initialize();
        return executor;
    }
}
```

11. CONCLUSION

This Low Level Design document provides comprehensive implementation details for the Loan Management System, including the newly added Gold Loan product. All five loan products (Personal, Business, Education, Vehicle, and Gold) follow the same core architecture while accommodating product-specific rules and workflows.

Key Implementation Highlights:

✓ Modular class design with clear separation of concerns

✓ Comprehensive database schema with proper indexing

✓ RESTful API design with consistent response formats

✓ Robust error handling and validation

✓ Security at every layer (JWT, encryption, RBAC)

✓ Performance optimizations (caching, async processing)

✓ Gold Loan with physical verification workflow. This LLD serves as the definitive guide for developers to implement the system with confidence and consistency.